



Mi Primera APP

API REST + Frontend en 90 minutos

FastAPI + React + Vite

Desarrollo de Software 2026 — UTN FRLP

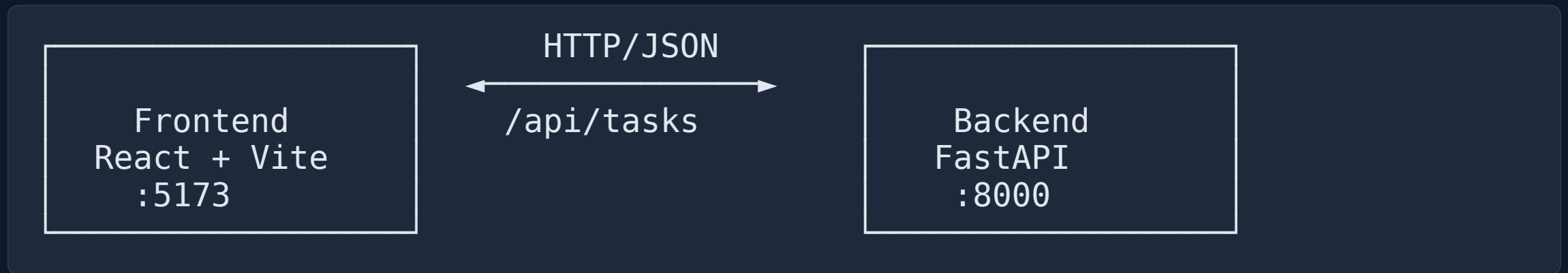
Objetivo del Taller

Construir una app de **Tareas** con API REST y frontend web.

- ▶  **Backend:** API REST con FastAPI en Python — 4 endpoints
- ▶  **Frontend:** Interfaz web con React — consume la API

*El foco NO es aprender frameworks. Es entender el **ciclo HTTP**.*

Arquitectura



Dos procesos independientes que se comunican por **HTTP**.

El frontend hace `fetch`, el backend responde con `JSON`.

La API — 4 Operaciones

Método	Ruta	Qué hace
GET	/api/tasks	Listar todas las tareas
POST	/api/tasks	Crear una tarea nueva
PATCH	/api/tasks/{id}	Marcar como hecha / desmarcar
DELETE	/api/tasks/{id}	Eliminar una tarea

Esto es un patrón **CRUD** — Create, Read, Update, Delete.

¿PATCH o PUT?

PATCH — Actualización parcial

Enviás SOLO el campo que cambia.

```
{ "completed": true }
```

PUT — Reemplazo completo

Enviás TODA la tarea.

```
{  
  "id": 1,  
  "title": "Comprar leche",  
  "completed": true,  
  "created_at": "2026-08-04T..."  
}
```

Usamos **PATCH** porque solo cambiamos un campo

Fase 1 — Instalación de Herramientas

Instalación de Herramientas

Herramienta	Para qué	Verificar
Python 3.12+	Lenguaje del backend	<code>python3 --version</code>
uv	Gestor de paquetes Python	<code>uv --version</code>
Node.js 22	Runtime de JavaScript	<code>node --version</code>
pnpm	Gestor de paquetes Node	<code>pnpm --version</code>

Ver [README.md](#) para instrucciones detalladas por SO.

uv — El gestor de paquetes Python

uv reemplaza `pip`, `venv` y `pip-tools` con un solo comando.

```
# Instalar uv (Linux / macOS)
curl -LsSf https://astral.sh/uv/install.sh | sh

# Crear proyecto
uv init mi-backend
cd mi-backend

# Agregar dependencias
uv add fastapi "uvicorn[standard]"

# Ejecutar
uv run main.py
```

~100ms vs los ~30s de pip 🏎️

Fase 2 — Crear el Backend

Crear el Backend

```
# Entrar al directorio
cd 01-mi-primer-app/backend

# Instalar dependencias
uv sync

# Ejecutar el servidor
uv run main.py
```

El servidor arranca en `http://localhost:8000`

 *No cierres esta terminal — necesitás el servidor corriendo.*

El Código — Modelos de Datos

```
class TaskCreate(BaseModel):  
    # Lo que el cliente ENVÍA (solo título)  
    title: str = Field(..., min_length=1, max_length=200)  
  
class Task(BaseModel):  
    # Lo que el servidor DEVUELVE (con ID y metadata)  
    id: int  
    title: str  
    completed: bool  
    created_at: str
```

¿Por qué dos modelos?

Separar entrada de salida es un patrón fundamental. El cliente no necesita enviar el ID — lo genera el servidor.

El Código — Endpoints

```
@app.get("/api/tasks")
def list_tasks():
    return tasks

@app.post("/api/tasks", status_code=201)
def create_task(body: TaskCreate):
    task = {"id": next_id, "title": body.title, "completed": False}
    tasks.append(task)
    return task

@app.patch("/api/tasks/{task_id}")
def toggle_task(task_id: int):
    task = find_task(task_id)
    task["completed"] = not task["completed"]
    return task
```

Fase 3 — Verificar la API

Verificar la API

Tres formas de probar que funciona:

 **Swagger UI**

<http://localhost:8000/docs> — Interfaz interactiva para probar cada endpoint.

 **Postman / Bruno**

Herramientas externas para testing de APIs.

 **curl**

```
curl -X POST http://localhost:8000/api/tasks \  
  -H "Content-Type: application/json" \  
  -d '{"title": "Hola"}'
```

Swagger UI — La Magia de FastAPI

Abrí <http://localhost:8000/docs> y probá:

```
# Crear tarea
POST /api/tasks
{ "title": "Aprender FastAPI" }

# Listar tareas
GET /api/tasks

# Toggle (marcar completada)
PATCH /api/tasks/1

# Eliminar
DELETE /api/tasks/1
```

Swagger genera la documentación automáticamente a partir del código.

Fase 4 — Crear el Frontend

Crear el Frontend

```
# Crear proyecto React con Vite
cd 01-mi-primer-app/frontend
pnpm create vite . --template react

# Instalar dependencias
pnpm install

# Ejecutar
pnpm dev
```

El frontend arranca en <http://localhost:5173>

*Con **ambos servidores corriendo**, abrí la app en el navegador.*

El Proxy de Vite

```
// vite.config.js
export default defineConfig({
  plugins: [react()],
  server: {
    proxy: {
      "/api": {
        target: "http://localhost:8000",
        changeOrigin: true,
      }
    }
  }
})
```

¿Qué hace?

Cuando el frontend hace `fetch("/api/tasks")`, Vite lo redirige a `localhost:8000/api/tasks`.

El frontend no sabe que el backend está en otro puerto. Simplifica todo

La Capa de Comunicación

```
// api.js – Toda la comunicación con el backend

const API_BASE = "/api";

async function request(url, options = {}) {
  const response = await fetch(`${API_BASE}${url}`, {
    headers: { "Content-Type": "application/json" },
    ...options,
  });
  return response.json();
}

export async function fetchTasks() { return request("/tasks"); }
export async function createTask(title) {
  return request("/tasks", {
    method: "POST",
    body: JSON.stringify({ title }),
  });
}
```

React — Lo que necesitás saber

useState

Guarda datos que cambian.

Ej: la lista de tareas.

useEffect

Ejecuta código al montar.

Ej: cargar tareas al iniciar.

JSX

HTML dentro de JavaScript.

Lo que ves en pantalla.

Eventos

React — El Flujo de Datos

```
Componente (App.jsx)
  |
  ▼ useState
Estado (tasks[])
  |
  ▼ .map()
UI Renderizada
```

React re-renderiza cuando el estado cambia.

Actualizás el estado → la UI se actualiza sola.

Fase 5 — Verificar la Integración

Verificar la Integración

Con ambos servidores corriendo:

1. Abrí `http://localhost:5173`
2. Escribí una tarea y hacé click en "Agregar"
3. Verificá que aparece en la lista
4. Marcala como completada (checkbox)
5. Eliminala (botón ×)

¿Funcionó? ¡Felicidades! Tu primera app fullstack. 🎉

El Ciclo de Vida del Software

Lo que hicimos hoy cubre las primeras 4 fases:

#	Fase	Qué hicimos
1	Requisitos	Definimos qué hace la app (4 endpoints)
2	Diseño	Arquitectura client-server, modelos
3	Implementación	Escribimos el código
4	Testing	Swagger, Postman, curl
5	Deployment	<i>(Futuro)</i> Docker + hosting
6	Mantenimiento	<i>(Futuro)</i> Features + fixes

Ejercicios para Llevar a Casa

● Comprensión

1. Abrí Swagger y contá cuántos endpoints hay.
2. Modificá el health check y recargá.
3. Eliminá `created_at` del modelo.

● Aplicación

4. Agregá campo `priority` (baja/media/alta).
5. Agregá endpoint de estadísticas.
6. Creá filtros en el frontend.

● Análisis



Construiste tu primera app fullstack en 90 minutos.

Backend	Frontend	Swagger
<code>uv run main.py</code>	<code>pnpm dev</code>	<code>localhost:8000/docs</code>

La Universidad te da el mapa. El recorrido lo hacés vos.